TMC Hackathon

Quantum Insurance Resource Allocations

Prepared by:

Max Anglade
Rami Chagnaud
Adel Mana

AXA X QOBLY Optimization Challenge

(February 27 – March 1, 2026)

Contents

0.1 Our Insurance Optimization Problem 1

0.2 Mathematical Modelling 1

0.2.1 Expected Gain per Claim 2

0.2.2 Budget Constraint 3

0.2.3 Network Structure and Clusters 4

0.2.4 Complete Objective Function 5

0.2.5 QUBO Formulation 5

0.2.6 Choice of Parameters λ_1 and λ_2 6

0.2.7 Conversion to Ising Hamiltonian 7

0.3 Proposed Algorithms 7

0.3.1 Cluster Generation 7

0.3.2 Classical Algorithm (Greedy) 8

0.3.3 Quantum-Classical Algorithm: Neural Quantum States . . . 8

0.3.4 Quantum Algorithm: VQE 10

0.3.5 Quantum Algorithm: QAOA 11

0.4 Discussion 16

0.4.1 Resource Estimation 16

0.4.2 Potential Advantages of Quantum Methods 16

0.4.3 Current Limitations of Quantum Methods 16

0.4.4 Complexity Analysis 17

0.4.5 Possible Model Extensions 17

0.4.6 Classical Benchmark Performance 18

Abstract

We address the problem of insurance fraud detection as a resource allocation problem under budget constraints, formulated as a Quadratic Unconstrained Binary Optimization (QUBO) problem and mapped to an Ising Hamiltonian. Our model incorporates a network structure through claim clustering and an economy-of-scale mechanism to capture the organized nature of insurance fraud. We implement and benchmark several optimization approaches: a classical greedy algorithm serving as a reference, a variational Neural Quantum States (NQS) method based on Restricted Boltzmann Machines, and quantum variational algorithms including VQE, QAOA, and its recursive variant RQAOA. On small instances (20 qubits), the greedy baseline achieves 93.6% of the optimal, while QAOA with warm-start matches this performance and the NQS approach surpasses it. Although current quantum methods do not yet outperform classical heuristics at small scales, our framework is designed for direct deployment on real quantum hardware, where its linear spatial complexity could unlock optimization at industrial scale.

0.1 Our Insurance Optimization Problem

During this 24-hour hackathon, we decided to work on the resource allocation problem for fraud detection.

We start from the following situation: a set of policyholders report a claim and request reimbursement from an insurance company. Among these clients, some commit fraud, whether intentionally or not. The insurance company can conduct investigations on the submitted claims, but each investigation costs the company a certain amount. Thus, the insurer faces a trade-off: it must detect as many fraudulent claims as possible to avoid being defrauded, while at the same time being constrained by a budget for claim verification.

Let us formalize all of this more precisely.

0.2 Mathematical Modelling

We consider an insurance company facing N claims from N of its clients. Its objective is to maximize its profit, or rather to minimize its loss, by detecting as

many fraudulent claims as possible, knowing that it can only investigate a limited number of cases.

Given a claim file i, we know that there is a probability P_i that the file is fraudulent. This probability P_i may depend on the type of claim but also on certain characteristics of the file under study.
We also know that this fraud can be intentional (to gain money) or unintentional (e.g., incorrect filling of administrative forms).
We consider that whether a fraud is intentional or not is given by a Bernoulli random variable V_i with parameter v_i, these variables being not necessarily independent a priori.

In addition to the v_i and P_i, we consider the C_i, the cost of investigating claim i, and M_i, the claimed amount.
The quantities v_i and P_i are known to insurance companies, which have very good fraud detection systems; we can mention the company Shift Technologies. We can then either use their data or generate other data using LLM models.

When there is fraud and it is intentional, we consider that if we detect it after an investigation, the person reimburses the cost of the investigation and also pays $M_i \times p$ where $p \geq 0$ is a multiplicative factor that can be seen as a fine. If, on the contrary, the fraud is unintentional and we have conducted the investigation, we consider that the person simply reimburses the cost of the investigation, without additional charges. Finally, if there is no fraud: if the insurer investigates the file, it pays C_i and reimburses M_i to the client, and if it does not investigate, it pays M_i.

We denote $x_i \in \{0, 1\}$ the decision variables indicating whether we decide to investigate or not.

0.2.1 Expected Gain per Claim

Using total probabilities and considering the disjoint events — there is no fraud, there is intentional fraud, there is unintentional fraud — we can write the expected gain for a claim i that is investigated ($x_i = 1$). Let us distinguish the three cases:

- **No fraud** (probability $1 - P_i$): the insurer pays the investigation cost C_i and reimburses M_i, resulting in a cost of $-(C_i + M_i)$.
- **Intentional fraud** (probability $P_i \cdot v_i$): the person reimburses the investigation cost and pays the fine $M_i \cdot p$, resulting in a gain of $+(C_i + M_i \cdot p + M_i) = C_i + M_i(1+p)$, since the insurer also recovers the amount M_i that it no longer has to reimburse.
- **Unintentional fraud** (probability $P_i \cdot (1 - v_i)$): the person reimburses the investigation cost and the claimed amount, resulting in a gain of $+(C_i + M_i)$.

If no investigation is conducted ($x_i = 0$), the insurer systematically pays M_i.

$$\tilde{\mathcal{L}} = \sum [x_i \cdot \big(P_i(v_i M_i p) - (1 - P_i)(C_i + M_i)\big) + (1 - x_i) \cdot M_i] \tag{1}$$

We wish to maximize this gain function, hence minimize $-\tilde{\mathcal{L}}$, and we reduce to a classical optimization problem.

By reordering the factors, factoring by the x_i, and removing the constants that unnecessarily complicate the computation, we obtain a new function to minimize:

$$\mathcal{L} = \sum_i x_i \cdot R_i \tag{2}$$

where R_i is the expected net gain from investigating claim i (compared to not investigating) and equals:

$$R_i = P_i\big(v_i \cdot M_i \cdot p + C_i + M_i\big) - \big(C_i\big) \tag{3}$$

The first term corresponds to the expected gain from the investigation (weighted by the probability of fraud and its intentional nature), while the second corresponds to the certain cost of the investigation and reimbursement. The higher R_i is, the more profitable it is to investigate claim i. For a negative R_i, the investigation will on average lose money for the company regardless of its outcome.

0.2.2 Budget Constraint

We are limited by a budget constraint. Considering only investigation expenses and introducing a budget B, we formally have:

$$\sum_{i=1}^{N} x_i C_i \leq B \tag{4}$$

To transform this inequality into an equality (necessary for the QUBO formulation), we introduce binary slack variables $z_0, z_1, \ldots, z_{M-1}$ with $M = \lfloor \log_2(B) \rfloor + 1$, such that:

$$\sum_{i=1}^{N} x_i C_i + \sum_{k=0}^{M-1} 2^k z_k = B \tag{5}$$

This constraint is imposed as a quadratic penalty in the objective function:

$$P_{\text{budget}} = \lambda_2 \left(\sum_{i=1}^{N} x_i C_i + \sum_{k=0}^{M-1} 2^k z_k - B \right)^2 \tag{6}$$

where $\lambda_2 > 0$ is a penalty parameter large enough so that any constraint violation is strongly discouraged.

0.2.3 Network Structure and Clusters

An observation caught our attention: insurance fraud is often carried out by organized rings. We must therefore introduce a network component into our modelling. Indeed, members of the same network will exhibit similar fraud behaviour due to their interactions.

We assume we have a correlation matrix $(b_{i,j})$ between the reported claims, notably induced by similar fraud probabilities. From this matrix, we construct K clusters $I_1, I_2, \ldots, I_K$ using a clustering algorithm.

We will encourage complete cluster investigation through the introduction of economies of scale. Let us introduce variables $y_c \in \{0, 1\}$ that equal 1 if we decide to investigate cluster c, and 0 otherwise. The benefit associated with activating cluster c is:

$$B_c = \left(\sum_{j \in I_c} C_j \right) \cdot \alpha \cdot \tanh\left(\frac{|I_c|}{k} \right) \tag{7}$$

where $\alpha > 0$ controls the magnitude of the economy of scale and $k > 0$ controls the saturation rate: a small k means that even small clusters benefit from a strong economy, while a large k reserves the advantage for large clusters.

We also wish to strongly correlate the variables y_c with the variables x_j within each cluster. That is, if a cluster is activated ($y_c = 1$), the uninvestigated claims in that cluster should be penalized. We add a term of the form:

$$P_{\text{cluster}} = \lambda_1 \sum_{c=1}^{K} \sum_{j \in I_c} (1 - x_j)\, y_c \tag{8}$$

This term equals zero when $y_c = 0$ (the cluster is not activated, no penalty) or when all $x_j = 1$ for $j \in I_c$ (the cluster is fully covered). It penalizes situations where a cluster is activated but some of its claims are not investigated.

0.2.4 Complete Objective Function

Gathering all terms, we obtain the objective function to minimize:

$$\boxed{L = \lambda_1 \sum_{c=1}^{K} \sum_{j \in I_c} (1 - x_j)\, y_c + \lambda_2 \left(\sum_{i=1}^{N} x_i C_i + \sum_{k=0}^{M-1} 2^k z_k - B \right)^2 - \sum_{i=1}^{N} x_i R_i - \sum_{c=1}^{K} y_c B_c} \tag{9}$$

The four terms play the following roles:

- $+\lambda_1 \sum_c \sum_{j \in I_c} (1 - x_j) y_c$: penalizes activated clusters whose claims are not all investigated.
- $+\lambda_2 (\cdots - B)^2$: enforces the budget constraint through quadratic penalty.
- $-\sum_i x_i R_i$: rewards investigating claims with high expected gain.
- $-\sum_c y_c B_c$: rewards cluster activation through the economy of scale.

0.2.5 QUBO Formulation

Denoting $\mathbf{v} = (x_1, \ldots, x_N,\ y_1, \ldots, y_K,\ z_0, \ldots, z_{M-1})^\top \in \{0,1\}^{N+K+M}$ the vector of all binary variables, the function L can be written in quadratic form:

$$L = \mathbf{v}^\top Q\, \mathbf{v} \tag{10}$$

where Q is a symmetric matrix of size $(N+K+M) \times (N+K+M)$. The function to optimize takes this unconstrained QUBO form thanks to our incorporation of the constraints into the function L.
This formulation is made possible by the property of binary variables $x_i^2 = x_i$, which allows encoding the linear terms on the diagonal of Q.

The coefficients of Q decompose as follows:

Diagonal. For the variables x_i:

$$Q_{x_i,x_i} = \lambda_2(C_i^2 - 2BC_i) - R_i$$

For the variables y_c:

$$Q_{y_c,y_c} = \lambda_1|I_c| - B_c$$

where $|I_c|$ is the cardinality of cluster c. For the variables z_k:

$$Q_{z_k,z_k} = \lambda_2(4^k - 2B \cdot 2^k)$$

Off-diagonal. The interactions x_j–y_c for $j \in I_c$:

$$Q_{x_j,y_c} = Q_{y_c,x_j} = -\frac{\lambda_1}{2}$$

The interactions x_i–x_j (from the budget term) for $i < j$:

$$Q_{x_i,x_j} = Q_{x_j,x_i} = \lambda_2 C_i C_j$$

The interactions x_i–z_k:

$$Q_{x_i,z_k} = Q_{z_k,x_i} = \lambda_2 C_i \cdot 2^k$$

The interactions z_k–z_l for $k < l$:

$$Q_{z_k,z_l} = Q_{z_l,z_k} = \lambda_2 \cdot 2^k \cdot 2^l$$

0.2.6 Choice of Parameters λ_1 and λ_2

The parameters λ_1 and λ_2 control the relative importance of the different terms in the objective function.

The parameter λ_2 must be large enough for the budget constraint to be satisfied. In practice, λ_2 is chosen so that the penalty for exceeding the budget by one unit is much larger than the maximum gain from any individual claim. However, a λ_2 that is too large makes the energy landscape very steep and complicates the optimization.

The parameter λ_1 governs the trade-off between cluster coverage and individual claim selection. A high λ_1 forces the algorithm to investigate all claims in an activated cluster, while a low λ_1 allows a more granular selection within clusters.

In practice, it is advisable to normalize the coefficients of the matrix Q and to adjust λ_1 and λ_2 through cross-validation or grid search to find the best trade-off.

0.2.7 Conversion to Ising Hamiltonian

To solve this problem on a quantum computer, we convert the QUBO formulation into an Ising Hamiltonian using the substitution:

$$x_i = \frac{1 - Z_i}{2} \tag{11}$$

where Z_i is the Pauli-Z operator acting on qubit i. This transformation produces a Hamiltonian of the form:

$$H = \sum_i h_i Z_i + \sum_{i<j} J_{ij} Z_i Z_j + \text{offset} \tag{12}$$

whose ground state encodes the optimal solution to our problem.

0.3 Proposed Algorithms

We decided to generate our datasets using LLMs in order to obtain the p_i, M_i, v_i, C_i and the distance matrix between two claims b_{ij}.
By generating these data through datasets, we can more easily modify them and test several possible scenarios.

0.3.1 Cluster Generation

We first implement a *k-medoids* algorithm to group the N claims into k clusters according to the distance matrix b_{ij}. This method operates directly on a distance matrix $B \in \mathbb{R}^{N \times N}$ and constrains each cluster center to be an actual data point — the *medoid* — defined as the member minimizing the sum of intra-cluster distances. At each iteration, points are assigned to the nearest medoid, then each medoid is recomputed as the element minimizing $\sum_{j \in C_c} B_{ij}$ within its cluster C_c. The algorithm converges when the medoids no longer change. To mitigate sensitivity to random initialization, n_{init} independent runs are performed and the partition minimizing the total cost $\sum_{i=1}^{N} B_{i,\,m(i)}$ is retained, where $m(i)$ denotes the medoid associated with point i. The per-iteration complexity is $\mathcal{O}(N^2)$, which remains reasonable for the sizes considered.

However, we realized that the number of clusters should not be imposed a priori but should be determined by an algorithm. Two solutions are available: either we set a distance threshold (defined on the claims and represented by the b_{ij}) beyond

which claims at distances below the threshold are grouped into clusters. Alternatively, we can also define an equivalence relation stating that vertices connected by edges below a threshold belong to the same cluster. We choose a threshold of 0.4 for our datasets.

Here is an example of clusters generated from a dataset.

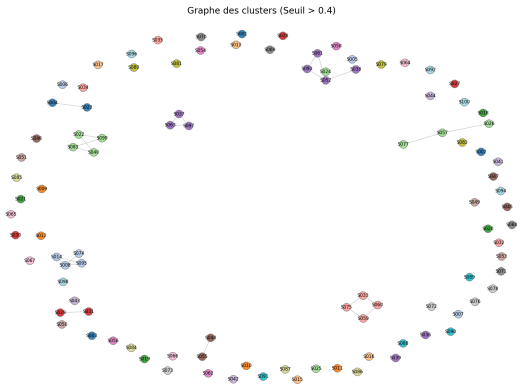

Figure 1: Networks with clusters from an LLM-generated dataset

0.3.2 Classical Algorithm (Greedy)

The idea is simple: at each step, we select the claim that maximizes the gain-to-cost ratio R_i/C_i among the remaining claims, as long as the budget allows. This algorithm runs in $\mathcal{O}(N \log N)$ and provides good-quality solutions for knapsack-type problems.

Its accuracy, however, depends on the problem size. At small sizes, the greedy solution often coincides with the exact optimum, making it a reliable reference for evaluating other methods. At large sizes, the algorithm may deviate from the optimum because it does not account for combinatorial interactions between claims. It nevertheless remains useful as a *warm start* for QAOA: by initializing the quantum state close to a good solution, it reduces the number of layers p needed to converge toward the Hamiltonian's ground state.

0.3.3 Quantum-Classical Algorithm: Neural Quantum States

Neural Quantum States (NQS) constitute a variational approach where the wave function is parameterized by a neural network. This method allows us to estimate the ground state energy and recover this configuration. We use a *Restricted Boltz-*

mann Machine (RBM), an architecture particularly suited to discrete quantum states, whose wave function is written as:

$$\log \Psi_{\mathrm{RBM}}(X) = \sum_{k=1}^{\alpha N} \log \cosh \left(b_k + \sum_{i=1}^{N} W_{ik} x_i \right)$$

where $\mathbf{x} \in \{-1, +1\}^N$ is a spin configuration, $\theta = \{b_k, W_{ik}\}$ are the variational parameters, and α is the ratio between the number of hidden neurons and input neurons. The ratio $\alpha = 2$ between hidden neurons and qubits provides sufficient expressiveness to capture the problem's correlations without over-parameterizing the model. The variational energy is estimated by Markov Chain Monte Carlo (MCMC) sampling, an essential method since the Hilbert space grows exponentially with N and cannot be exhaustively explored. The parameters are optimized by Stochastic Reconfiguration (SR), which plays the role of a natural gradient by taking into account the geometry of the variational space, offering much more stable convergence than a simple SGD. The diagonal shift $\epsilon = 10^{-3}$ regularizes this matrix to avoid numerical instabilities. After convergence in $n_{\mathrm{iter}} = 500$ iterations, we sample 5000 configurations from the variational state, which we project into $\{0, 1\}^N$ to extract the lowest-energy classical solution $\mathbf{x}^* = \arg\min_{\mathbf{x}} \mathbf{x}^\top Q\, \mathbf{x}$.

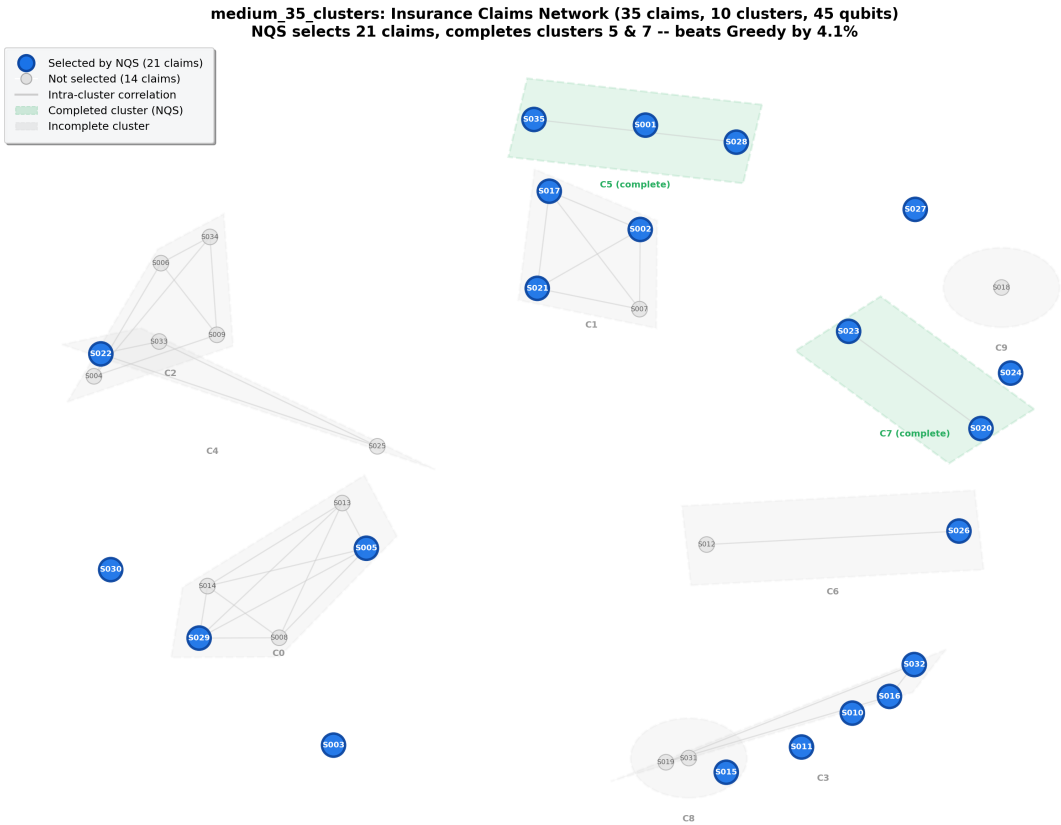

Figure 2: Graph representation of the result with a constrained NQS

0.3.4 Quantum Algorithm: VQE

The VQE (*Variational Quantum Eigensolver*) algorithm is a hybrid quantum-classical algorithm that seeks to minimize the energy $\langle\psi(\boldsymbol{\theta})|H|\psi(\boldsymbol{\theta})\rangle$ by optimizing the parameters $\boldsymbol{\theta}$ of a parametric quantum circuit (ansatz).

The circuit we use is an ansatz composed of alternating layers of $R_Y(\theta_i)$ rotations on each qubit and CNOT gates in a circular chain for entanglement. The circuit depth (number of layers) controls the expressiveness of the ansatz.

Parameter optimization is performed by a classical optimizer (Adam or COBYLA). Gradients are computed via the *parameter-shift rule*. For simulation, we use PennyLane's `lightning.gpu` backend, which leverages NVIDIA's cuQuantum library to simulate the state vector on GPU, allowing us to handle instances up to 25 qubits. However, the VQE method, which seemed natural to us due to its Hamiltonian minimization, is in practice not viable since the optimization with 23 qubits would take approximately 5 hours. This computation time prevents us from running mul-

tiple different tests on the algorithm to optimize hyperparameters or study their impact on convergence and computation time.

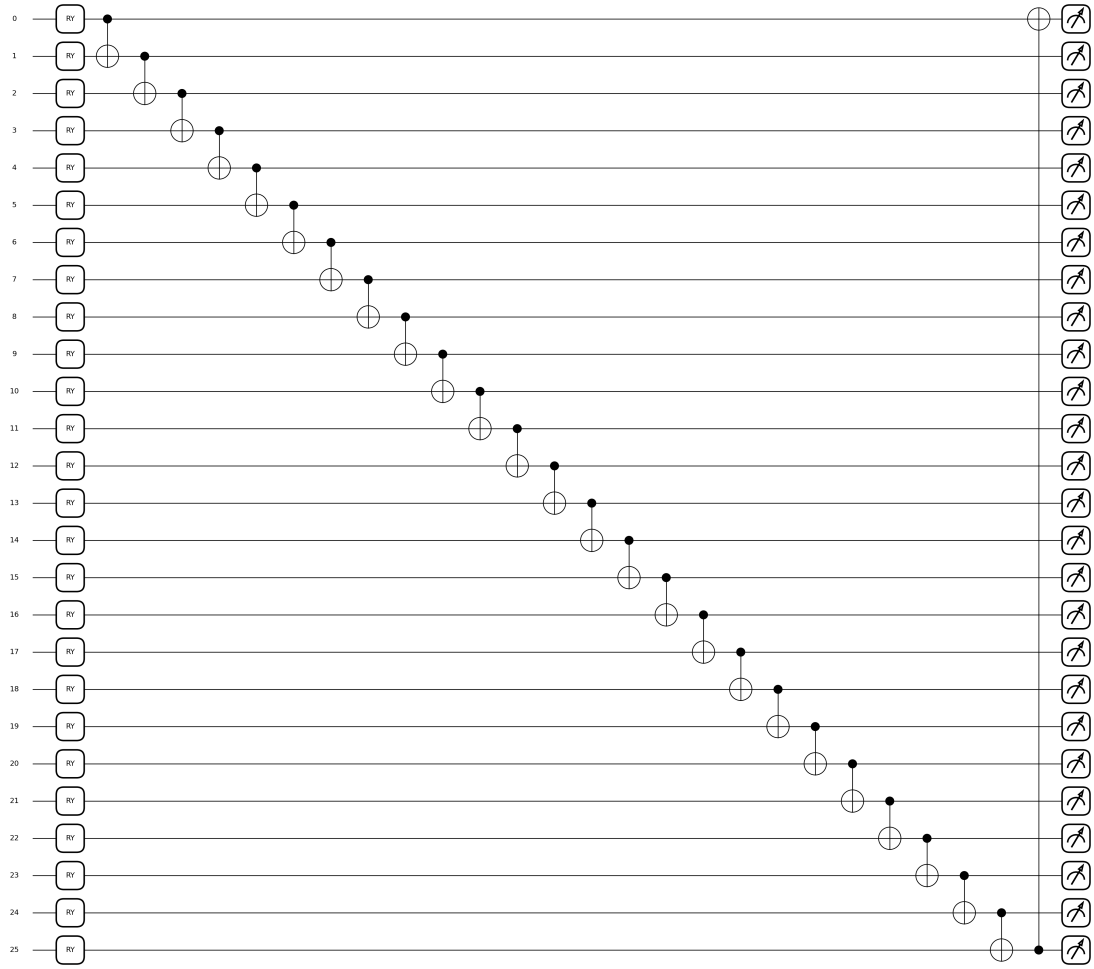

Figure 3: Quantum circuit of the VQE we used

0.3.5 Quantum Algorithm: QAOA

QAOA Principle

To address this excessive computation time problem with VQE, we now use a *Quantum Approximate Optimization Algorithm* (QAOA). A QAOA algorithm is a variational algorithm specifically designed for combinatorial optimization problems with variables taking values in $\{0, 1\}$. Unlike VQE, which uses a generic ansatz, QAOA leverages the problem structure by alternating two unitary operators:

- The **cost operator** $U_C(\gamma) = e^{-i\gamma H_C}$, where H_C is the problem Hamiltonian.

- The **mixer operator** $U_M(\beta) = e^{-i\beta H_M}$, where $H_M = \sum_i X_i$ promotes exploration of the solution space.

The final state after p layers is:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = U_M(\beta_p)U_C(\gamma_p)\cdots U_M(\beta_1)U_C(\gamma_1)|+\rangle^{\otimes n} \tag{13}$$

The parameter p (depth) controls the approximation quality: theoretically, on a simulator where noise does not come into play, $p \to \infty$ converges to the exact solution. In practice, we test different values of p.

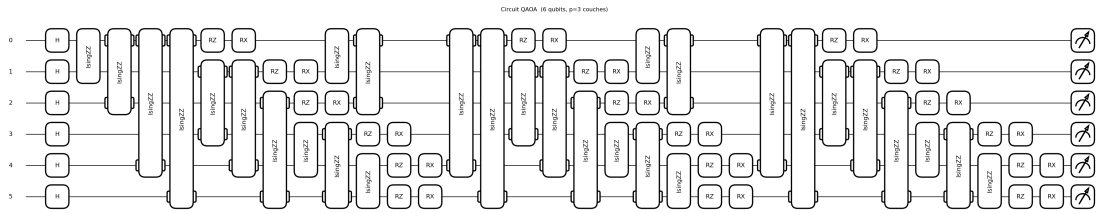

Figure 4: Quantum circuit of the RQAOA we used with $n_{\text{qubits}} = 6$ and $p = 3$

Basic QAOA

Our first implementation uses standard QAOA with an Adam optimizer and 300 optimization steps at depth $p = 8$. This version serves as a starting point and reaches approximately 47% of the brute-force optimal on our test instance (20 qubits, budget $B = 20{,}000$). This limited performance is expected: random parameter initialization can trap the optimizer in a poor local minimum.

Warm-Start QAOA

To address the initialization problem, we use the greedy solution as a *warm start*. Specifically, we initialize the circuit state not at $|+\rangle^{\otimes n}$ but at a state close to the greedy solution, using a parameter ε that controls the perturbation. With $p = 8$, $\varepsilon = 0.3$, and 300 optimization steps, this method reaches 93.6% of the optimum, matching the greedy algorithm's performance. This validates the effectiveness of the warm start but does not tell us whether our algorithm is truly performant, since we directly initialized the optimization at the greedy solution. Moreover, it fails to find a better solution starting from the greedy solution, so the warm start is not truly conclusive.

Multi-Start QAOA

The *multi-start* approach consists of relaunching the QAOA optimization multiple times with different initializations and keeping the best solution found. This allows us to explore the configuration space more thoroughly. We then test two configurations:

- **Without warm start** (5×60 steps, $p = 8$): reaches 76.4% of the optimum.
- **With warm start** (5×100 steps, $\varepsilon = 0.5$): reaches 93.6% of the optimum.

We observe that the warm start improves performance, but this was expected given the greedy algorithm's efficiency at small sizes. However, we also observe that the multi-start does not allow us to "escape" the greedy solution and find a better one, which is our main objective.

RQAOA (Recursive QAOA)

RQAOA is a variant of QAOA that iteratively reduces the problem size by fixing the most correlated variables after each QAOA execution. This allows reducing the number of required qubits while progressively guiding toward a good solution.

We tested several configurations:

- **Best configuration** ($p = 15$, $\lambda_2 : 0 \to 4$ progressive, 5 starts, warm initialization on y and z): reaches 84.8% of the optimum in 1365 seconds. The progressive annealing of λ_2 (starting from a low value for exploration, then increasing to enforce the budget constraint) proves beneficial. Moreover, even though the percentages of the greedy and our solution appear far apart, at small sizes they correspond to only one claim difference in our optimization.
- **Intermediate configurations** ($p = 8$, 200 steps, $\lambda_2 = 4$): 76.9% in 445 seconds.
- **Lightweight configurations** ($p = 3$, 50 steps): 67.1% in 48 seconds, a good time/quality trade-off.

Comparative Results

Figure 5 summarizes the performance of all approaches tested on the `small_12` instance (20 qubits, $B = 20{,}000$). The optimum is, as we expected at small sizes,

the solution obtained by brute force at $L^* = 48.78$.

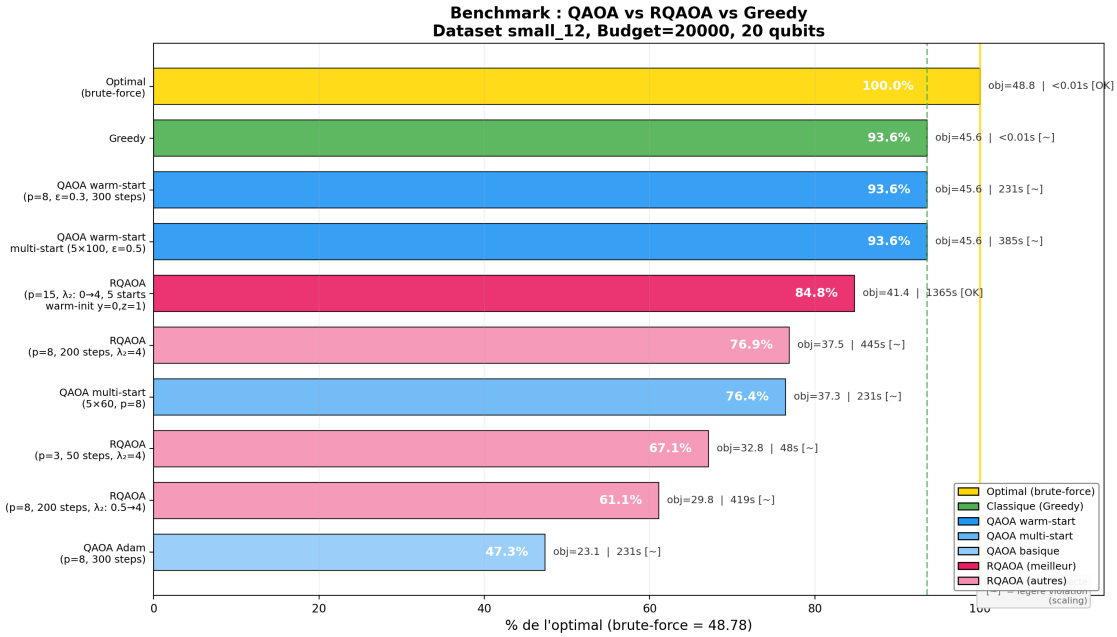

Figure 5: Comparative benchmark QAOA vs RQAOA vs Greedy on 20 qubits. Bars indicate the percentage of the optimal objective achieved. The symbol [~] indicates a slight budget constraint violation (scaling).

Several lessons emerge from this benchmark:

(i) The **classical greedy** algorithm at small sizes is remarkably performant (93.6% of the optimum) and nearly instantaneous. It is a difficult reference to beat.

(ii) The **warm-start QAOA** matches the greedy (93.6%), showing that the quantum circuit efficiently exploits the classical initialization, but fails to significantly deviate from it.

(iii) The **RQAOA** with λ_2 annealing reaches 84.8%, below the warm start but with the advantage of better respecting the constraints.

(iv) **Initialization** is crucial: basic QAOA (Adam, random initialization) only reaches 47.3%.

(v) Most quantum methods exhibit a slight budget constraint violation [~], suggesting that the parameter λ_2 requires careful tuning.

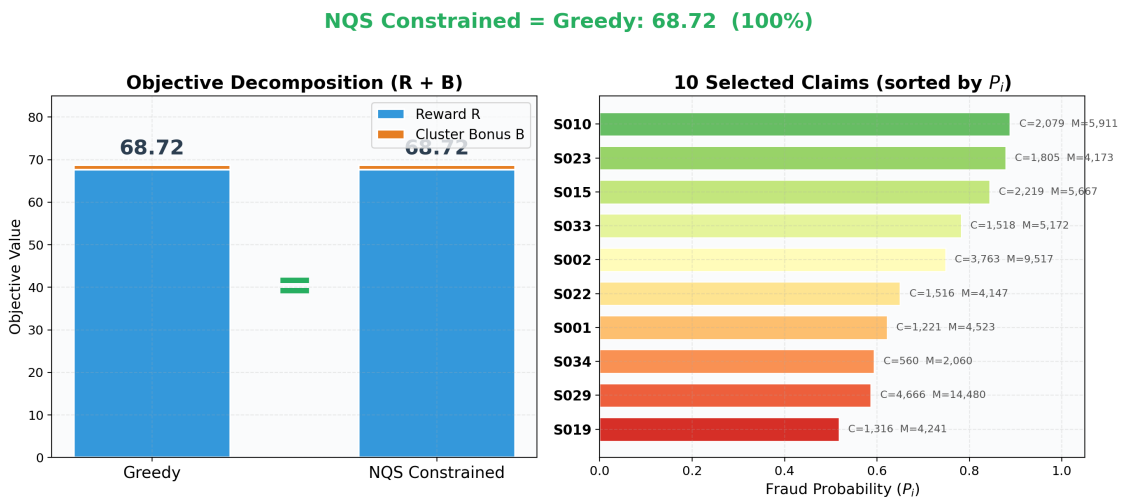

Figure 6: Greedy methods vs NQS Constrained

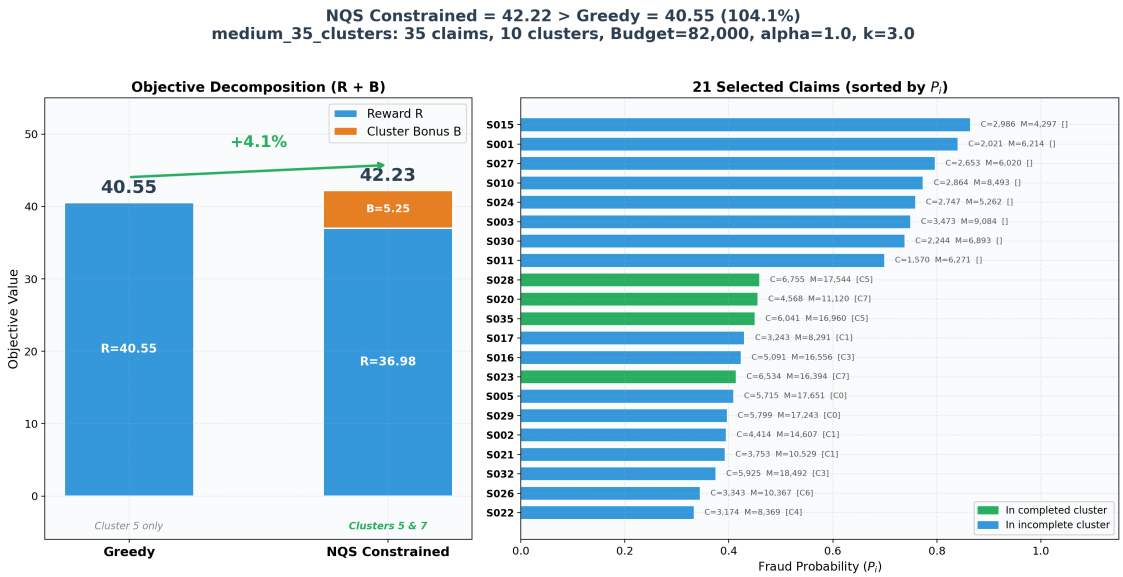

Figure 7: Greedy methods vs NQS Constrained

This shows that with all the optimization we used, we were able to produce a result through the NQS method that beat the Greedy methods for our problem.

0.4 Discussion

0.4.1 Resource Estimation

Number of qubits. The total number of qubits required is:

$$n_{\text{qubits}} = N + K + M = N + K + \lfloor \log_2(B) \rfloor + 1 \tag{14}$$

For our test instance: $N = 12$ claims (the x_i), $K = 5$ clusters (the y_i), $M = 3$ slack bits (the z_i), giving 20 qubits. For a realistic instance of 50 claims with 43 clusters and $B = 50{,}000$, we reach 109 qubits.

Circuit depth. For QAOA at depth p, each layer requires the application of U_C and U_M. The cost operator U_C contains a number of gates proportional to the number of non-zero terms in the Hamiltonian, i.e., $O(N^2 + NK + NM)$. The total depth is therefore $O(p \cdot (N^2 + NK + NM))$. For VQE with an ansatz of L layers, the depth is $O(L \cdot n_{\text{qubits}})$.

0.4.2 Potential Advantages of Quantum Methods

Quantum methods could offer an advantage in the following situations:

- **Scaling to large sizes**: for NP-hard combinatorial problems like ours, quantum methods could offer a polynomial speedup over classical exhaustive search, particularly when the energy landscape exhibits structures exploitable by quantum algorithms.
- **Strongly correlated problems**: the cluster structure of our problem, with correlations between the variables x_j and y_c, could be naturally captured by quantum entanglement and thus better handled by our quantum algorithms.

0.4.3 Current Limitations of Quantum Methods

Our experiments highlight several limitations:

- **The greedy algorithm is hard to beat**: on our test instance, the simple greedy algorithm ($O(N \log N)$) already reaches 93.6% of the optimum. Quantum methods do not surpass it. However, this could be due to the small size of our system: at larger sizes, our algorithm should perform better.

- **Sensitivity to hyperparameters**: QAOA performance strongly depends on p, the initialization, the number of optimization steps, and especially the tuning of λ_2. The annealing of λ_2 (RQAOA) is promising but time-consuming.
- **Constraint violation**: the quadratic penalty approach does not guarantee strict budget compliance. Most quantum solutions exhibit a slight violation.
- **Scalability**: statevector simulation (even on GPU) is limited to approximately 30 qubits ($2^{30} \approx 10^9$ amplitudes $\times$ 16 bytes $\approx$ 16 GB VRAM). It would be necessary to analyze the results for large system sizes and in particular readjust the hyperparameters λ_2 and λ_1.
- **Quantum noise**: on real hardware, quantum gates are noisy, which degrades solution quality, especially for deep circuits (high p).

0.4.4 Complexity Analysis

The resource allocation problem under budget constraint is a generalized knapsack problem, which is NP-hard. The search space grows exponentially: 2^{n_q} (where $n_q = K + N + M$) possible configurations.

Method	**Time Complexity**	**Space Complexity**
Brute-force	$O(2^{n_q})$	$O(n_q)$
Greedy	$O(N \log N)$	$O(N)$
NQS	$O(K \cdot \alpha \cdot N^2)$	$O(\alpha \cdot N^2)$
VQE (simulation)	$O(S \cdot p \cdot 2^{n_q})$	$O(2^{n_q})$
QAOA (simulation)	$O(S \cdot p \cdot 2^{n_q})$	$O(2^{n_q})$
QAOA (hardware)	$O(S \cdot p \cdot n_q^2)$	$O(n_q)$

where S is the number of optimization steps and p the circuit depth.

On real quantum hardware, the space complexity is linear in n_q (one physical qubit per variable), which constitutes the fundamental advantage of the quantum approach over classical simulators that require exponential memory.

0.4.5 Possible Model Extensions

Several extension avenues are conceivable:

- **Dynamic model**: process claims in discrete temporal waves, with updating of probabilities $P_i(n)$ as investigations are completed.
- **Adaptive clustering**: refine clusters based on investigation results.
- **Problem-informed ansatz**: in quantum algorithms, instead of a generic ansatz, design an ansatz that respects the problem structure as in the solution proposed by our Neural Quantum Networks (for example, entanglement layers limited to nearest neighbours).
- **Alternative quantum algorithms**: explore other architectures to solve our optimization problem, such as Quantum Annealing (D-Wave).
- **Industrial-scale deployment**: for hundreds of claims, combine a classical pre-selection (filtering by R_i/C_i) with quantum optimization on the reduced subset.

0.4.6 Classical Benchmark Performance

For 12 claims, corresponding to about twenty qubits, the greedy algorithm, with a complexity of $O(N \log N)$, reaches 93.6% of the optimum in less than 0.01 seconds. This makes it an extremely competitive reference at small sizes. Dynamic programming would yield the exact optimum in $O(NB)$ but becomes prohibitive when B is large.

Our benchmark shows that quantum methods, in their current state of development (classically simulated), do not surpass the greedy algorithm on small instances. However, the interest of quantum approaches lies in their scaling potential: while exact classical algorithms become impractical beyond a few dozen variables, quantum algorithms on real hardware could maintain linear space complexity, paving the way for optimization of industrial-scale instances.

Conclusion

We have formulated the resource allocation problem for insurance fraud detection as a QUBO problem, integrating a network structure through clusters and a budget constraint. The conversion to an Ising Hamiltonian allows us to apply variational quantum algorithms (VQE and QAOA) to this problem.

Our experiments show that QAOA with classical warm start achieves the best quantum performance (93.6% of the optimum), matching the classical greedy algorithm. RQAOA with progressive annealing of λ_2 offers an original approach for constraint satisfaction. The main current limitations are scalability (limited to 30 qubits in GPU simulation) and the difficulty of surpassing classical heuristics on small instances.

The interest of this approach lies in its direct transferability to real quantum hardware when it becomes available at the required scale, as well as in the richness of the QUBO framework, which naturally allows the integration of constraints and complex correlation structures.